Ex No: 08 Date : 10.03.2026	CONVERSION OF BNF GRAMMAR TO YACC AND GENERATION OF ABSTRACT SYNTAX TREE USING LEX AND YACC

AIM:

To convert BNF grammar into YACC form and generate an Abstract Syntax Tree (AST) for arithmetic expressions using Flex and Bison.

ALGORITHM:

Step 1: Start the program.

Step 2: Define the tokens using Lex for recognizing numbers, operators (+, *), and parentheses.

Step 3: When a number is detected using the pattern [0-9]+, convert it into an integer using atoi() and return the token NUMBER to the parser.

Step 4: Ignore whitespace characters such as spaces and tabs during lexical analysis.

Step 5: Pass all recognized tokens from the lexical analyzer to the parser implemented using Yacc.

Step 6: Define a structure called node to represent the Abstract Syntax Tree (AST) node with fields:

- operator (op)
- value (value)
- left child pointer
- right child pointer

Step 7: Create a function makeLeaf() to create a leaf node when a numeric value is encountered.

Step 8: Create another function makeNode() to create an internal node when an operator (+, *) is encountered and connect its left and right children.

Step 9: Define grammar rules in YACC for parsing arithmetic expressions:

expr → expr + term | term
term → term * factor | factor
factor → NUMBER | (expr)

Step 10: While parsing the input expression, construct the AST:

- If a number is found, create a leaf node.

- If an operator rule is matched, create an internal node connecting the left and right operands.

Step 11: Store the final generated node as the root of the Abstract Syntax Tree.

Step 12: After successful parsing, call the printTree() function to display the AST.

Step 13: Traverse the tree recursively and print each node with indentation to show the tree structure.

Step 14: Display the generated Abstract Syntax Tree in the terminal.

Step 15: Stop the program.

PROGRAM:

Lex Program (lexer.l):

```
%{
#include "parser.tab.h"
#include <stdlib.h>
%}

%%

[0-9]+   { yylval.num = atoi(yytext); return NUMBER; }
[ \t]    ;
\n       return 0;
.        return yytext[0];

%%

int yywrap(){
   return 1;
}
```

YACC Program (parser.y):

```
%{
#include<stdio.h>
#include<stdlib.h>

typedef struct node{
   char op;
   int value;
   struct node *left,*right;
}node;

node *root;
```

```
node* makeNode(char op,node* l,node* r){
    node* n=(node*)malloc(sizeof(node));
    n->op=op;
    n->left=l;
    n->right=r;
    return n;
}

node* makeLeaf(int val){
    node* n=(node*)malloc(sizeof(node));
    n->value=val;
    n->left=n->right=NULL;
    return n;
}

void printTree(node* t,int level){
    if(t==NULL)
        return;

    for(int i=0;i<level;i++)
        printf("   ");

    if(level>0)
        printf("|--");

    if(t->left==NULL && t->right==NULL)
        printf("%d\n",t->value);
    else
        printf("%c\n",t->op);

    printTree(t->left,level+1);
    printTree(t->right,level+1);
}

void yyerror(char *s){
    printf("Error: %s\n",s);
}

int yylex();

%}

%union{
    int num;
    struct node* ptr;
}

%token <num> NUMBER
%type <ptr> expr term factor
```

```
%%

expr : expr '+' term { $$ = makeNode('+',$1,$3); root=$$; }
     | term          { $$ = $1; root=$$; }
     ;

term : term '*' factor { $$ = makeNode('*',$1,$3); }
     | factor          { $$ = $1; }
     ;

factor : NUMBER        { $$ = makeLeaf($1); }
       | '(' expr ')'  { $$ = $2; }
       ;

%%

int main(){
    printf("Enter expression:\n");
    yyparse();

    printf("\nAbstract Syntax Tree:\n");
    printTree(root,0);

    return 0;
}
```

OUTPUT:

```
PS C:\Users\ELCOT\OneDrive\Desktop\OneDrive\Documents\BNF> cd "C:\Users\ELCOT\OneDrive\Desktop\OneDrive\Documents\BNF"
PS C:\Users\ELCOT\OneDrive\Desktop\OneDrive\Documents\BNF> bison -d parser.y
PS C:\Users\ELCOT\OneDrive\Desktop\OneDrive\Documents\BNF> flex lexer.l
PS C:\Users\ELCOT\OneDrive\Desktop\OneDrive\Documents\BNF> gcc parser.tab.c lex.yy.c -o ast.exe
PS C:\Users\ELCOT\OneDrive\Desktop\OneDrive\Documents\BNF> .\ast.exe

Enter expression:
a+b*c

Abstract Syntax Tree:
+
   i
   *
      i
      i
```

Algorithm & Design	20	
Implementation & Output	20	
Viva	20	
Record	15	
Total	75	

Result:

Thus the BNF grammar was successfully converted into YACC form and the Abstract Syntax Tree was generated using Lex and Yacc.

Ex

Ex No: 09 Date : 10.03.2026	GENERATION OF THREE ADDRESS CODE FOR ARITHMETIC EXPRESSIONS USING LEX AND YACC

AIM:

To design a program that generates Three Address Code (TAC) for arithmetic expressions using Flex and Bison.

ALGORITHM:

Step 1: Start the program.

Step 2: Define tokens using Lex for identifiers, numbers, operators (+, *), and parentheses.

Step 3: When an identifier [a-zA-Z] or number [0-9]+ is detected, pass it to the parser using yylval.

Step 4: Ignore whitespace characters such as spaces and tabs.

Step 5: Pass the tokens from the lexical analyzer to the parser created using Yacc.

Step 6: Define grammar rules for arithmetic expressions:

expr → expr + term | term
term → term * factor | factor
factor → ID | NUM | (expr)

Step 7: Maintain a temporary variable counter (t1, t2, t3...) to store intermediate results.

Step 8: When an expression rule is matched, generate a new temporary variable using the newTemp() function.

Step 9: Print the corresponding Three Address Code instruction in the format:

temp = operand1 operator operand2

Step 10: Store the temporary variable as the result for the next computation.

Step 11: Continue parsing until the entire input expression is processed.

Step 12: Display all generated Three Address Code instructions.

Step 13: Stop the program.

PROGRAM:

Lex Program (lexer.l):

```
%{
#include "parser.tab.h"
#include <string.h>
%}

%%

[a-zA-Z]     { yylval.str = strdup(yytext); return ID; }
[0-9]+       { yylval.str = strdup(yytext); return NUM; }
[ \t]        ;
\n           return 0;
.            return yytext[0];

%%

int yywrap(){
    return 1;
}
```

Yacc Program (parser.y):

```
%{
#include<stdio.h>
#include<stdlib.h>
#include<string.h>

int temp_count = 1;

char* newTemp(){
    char *temp = (char*)malloc(10);
    sprintf(temp,"t%d",temp_count++);
    return temp;
}

void yyerror(char *s){
    printf("Error: %s\n",s);
}

int yylex();

%}

%union{
    char* str;
}

%token <str> ID NUM
%type <str> expr term factor

%%
```

```
expr : expr '+' term
     {
       char *temp = newTemp();
       printf("%s = %s + %s\n",temp,$1,$3);
       $$ = temp;
     }
     | term { $$ = $1; }
     ;

term : term '*' factor
     {
       char *temp = newTemp();
       printf("%s = %s * %s\n",temp,$1,$3);
       $$ = temp;
     }
     | factor { $$ = $1; }
     ;

factor : ID  { $$ = $1; }
       | NUM { $$ = $1; }
       | '(' expr ')' { $$ = $2; }
       ;

%%

int main(){
    printf("Enter expression:\n");
    yyparse();
    return 0;
```

OUTPUT:

```
PS C:\Users\ELCOT\OneDrive\Desktop\OneDrive\Documents\Three Adress code> cd "
"C:\Users\ELCOT\OneDdve\Desktop\OneDrive\Documents\Three Adress code>
PS C:\Users\ELCOT\OneDrive\Desktop\OneDrive\Documents\Three Adress code>
PS C:\Users\ELCOT\OneDrive\Desktop\OneDrive\Documents\Three Adress code>
PS C:\Users\ELCOT\OneDrive\Desktop\OneDrive\Documents\Three Adress ty
PS C:\Users\ELCOT\OneDrive\Desktop\OneDrive\Documents\Three Adrees.tab.c lex.yy.c -o tac.exe
PS C:\Users\ELCOT\OneDrive\Desktop\OneDrive\Documents\Three Adress code> .

Enter expressions (press Ctrl+Z then Enter to quit):
a+b*c
Three Adress Code:
t0 = b * c
t1 = a + t0
^Z

PS C:\Users\ELCOT\OneDrive\Desktop\OneDrive\Documents\Three Adress code>
```

Algorithm & Design	20	
Implementation & Output	20	
Viva	20	
Record	15	
Total	75	

RESULT:

Thus the Three Address Code for the given arithmetic expression was successfully generated using Lex and Yacc.